The Quantum Fourier Transform with Python and Qiskit

A complete study guide: Fourier analysis from scratch, roots of unity, the QFT formula and circuit,
full analysis of the 5-qubit experiment, tested code, period-finding demo and viva questions

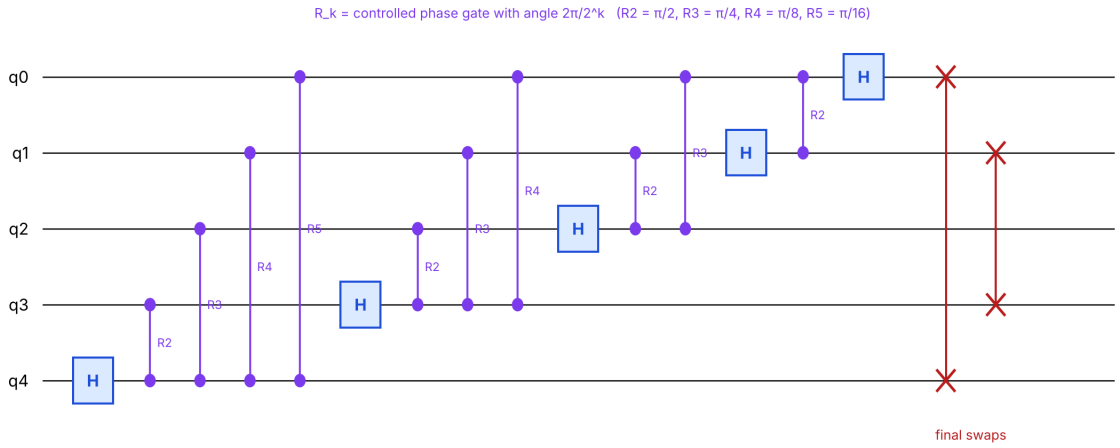


Figure 1: Structure of the 5-qubit QFT: Hadamards, controlled-phase rotations R_k, then qubit-order swaps.

Python • Qiskit • Qiskit Aer • Fourier analysis • Phase • Interference

Contents

1. Aim

2. Fourier transform basics (classical background)

3. Complex numbers and roots of unity

4. The Quantum Fourier Transform: definition and examples

5. Product form and the QFT circuit

6. Qiskit bit order

7. Analysis of your experiment: what output should we expect?

8. Your code explained line by line

9. Complete code and real output

10. Modern code (QFTGate) checked against theory

11. Building the QFT yourself and verifying it

12. Period finding: where the QFT is really useful

13. Applications and honest limitations

14. Troubleshooting

15. Viva questions and answers

16. Glossary

17. Result and conclusion

> **Important**
> **Things to know before you start.** (1) Your code runs correctly. I executed it. (2) In current Qiskit (2.1 and later) the `QFT` class it imports is **deprecated** and will be removed in Qiskit 3.0. It still works but prints a warning; Section 10 shows the modern `QFTGate` version. (3) The "expected output" in the notes lists bitstrings with "...". The real distribution is not uniform: it is a smooth bell-like shape with a hole at 16 (10000) that Section 7 derives exactly, so your result can be checked against theory.

1. Aim

To implement a 5-qubit Quantum Fourier Transform (QFT) in Python using Qiskit, apply it to a prepared input state, measure the result on a simulator, and understand and verify the output distribution mathematically.

2. Fourier Transform Basics (Classical Background)

2.1 Time domain and frequency domain

A signal such as a sound wave can be described in two equivalent ways: how its value changes over **time**, or which **frequencies** it contains and how strong each is. The **Fourier transform** converts between these two descriptions. A pure musical note looks like a complicated wiggle in time but is a single spike in the frequency picture.

2.2 Discrete Fourier Transform (DFT)

For N numbers $x_0, \ldots, x_{N-1}$, the DFT produces N numbers:

$$X_k = \Sigma_{j=0}^{N-1} x_j \, e^{2\pi i j k/N}$$

(Textbooks differ on the sign in the exponent. The QFT uses the + sign shown here, and a factor $1/\sqrt{N}$ is added so that the transform is unitary.)

Worked example, N = 4. Take the signal $(x_0, x_1, x_2, x_3) = (1, 0, -1, 0)$. This is one full cosine wave sampled four times. Using $\omega = e^{2\pi i/4} = i$: $X_k = 1 \cdot \omega^0 + (-1) \cdot \omega^{2k} = 1 - (-1)^k$. So $(X_0, X_1, X_2, X_3) = (0, 2, 0, 2)$. The spikes at k = 1 and k = 3 say “frequency 1” (and its mirror image) is present.

2.3 Cost

Computing the DFT directly takes about N^2 operations. The Fast Fourier Transform (FFT) reduces this to about N log N. For $N = 2^n$ that is roughly $n \cdot 2^n$ operations. Keep this number in mind: the QFT circuit will need only about n^2 gates (Section 5), but with an important catch (Section 13).

3. Complex Numbers and Roots of Unity

3.1 Euler's formula

$$e^{i\theta} = \cos\theta + i\sin\theta$$

The number $e^{i\theta}$ sits on the unit circle at angle θ. Its magnitude is 1. Multiplying by $e^{i\theta}$ rotates a complex number by θ and does not change its length. In quantum computing this rotation is called a **phase**.

3.2 Roots of unity

Let $N = 2^n$ and $\omega = e^{2\pi i/N}$. Then $\omega^N = e^{2\pi i} = 1$, so ω is an N-th root of unity, and the powers ω^0, ω^1, …, ω^{N-1} are N points equally spaced around the unit circle.

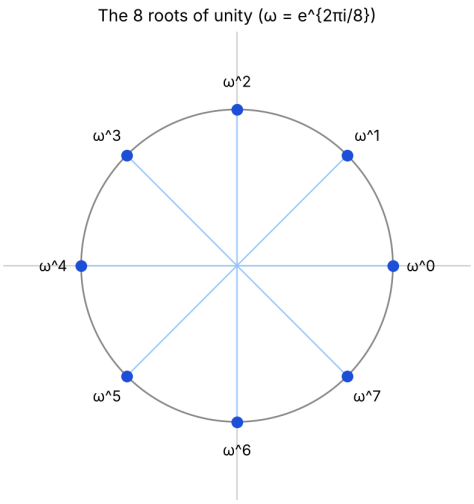


Figure 2: The eight 8th roots of unity. Each step ω multiplies by a rotation of 360°/8 = 45°.

Useful facts: $\omega^a \cdot \omega^b = \omega^{a+b}$; $\omega^N = 1$ so exponents can be reduced modulo N; and for N = 32, $\omega^{16} = -1$ (half a turn).

3.3 Why phases matter for measurement

Measuring a qubit in the computational basis reveals only $|\text{amplitude}|^2$. Phases are invisible to that measurement, but they control how amplitudes **interfere** when added. The QFT stores information in phases, which is why its output often looks uniform for a single input.

4. The Quantum Fourier Transform: Definition and Examples

Definition
The QFT on n qubits ($N = 2^n$) is the unitary operation defined on basis states by

$\mathrm{QFT}|x\rangle = (1/\sqrt{N}) \sum_{y=0}^{N-1} e^{2\pi i xy/N} |y\rangle$.

On a general state $\sum_x a_x|x\rangle$ it maps the vector of amplitudes (a_x) to its DFT (with $1/\sqrt{N}$ normalisation).

4.1 Matrix form

The matrix has entries $F_{yx} = \omega^{xy}/\sqrt{N}$, where x labels columns and y rows. It is unitary, so the QFT is reversible: the **inverse QFT** uses ω^{-xy}.

N = 2 (one qubit). $\omega = -1$, so

$$F_2 = (1/\sqrt{2}) \times \begin{vmatrix} 1 & 1 \\ 1 & -1 \end{vmatrix} = \mathrm{H}, \text{ the Hadamard gate.}$$

So the 1-qubit QFT is exactly the Hadamard gate.

N = 4 (two qubits). $\omega = i$, so

$$F_4 = \tfrac{1}{2} \times \begin{vmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{vmatrix}$$

Worked example: $\mathrm{QFT}|1\rangle$ on two qubits is the second column: $\tfrac{1}{2}(|0\rangle + i|1\rangle - |2\rangle - i|3\rangle)$. All four outcomes have probability 1/4, but the phases 1, i, −1, −i are different and encode x = 1.

4.2 An important observation

Important
For **any single basis state** $|x\rangle$, every outcome y has the same probability 1/N, because $|\omega^{xy}/\sqrt{N}|^2 = 1/N$. So measuring right after $\mathrm{QFT}|x\rangle$ shows a flat distribution, whatever x is. The information about x sits in the phases and is invisible. Only when several inputs interfere (a superposition) does the measured distribution develop structure. This is exactly what happens in your experiment.

4.3 Basis change

The QFT changes from the computational basis $\{|x\rangle\}$ to the **Fourier basis** $\{\mathrm{QFT}|x\rangle\}$. The states $\mathrm{QFT}|x\rangle$ are equal-magnitude superpositions that differ only in phase. Applying the inverse QFT converts back.

5. Product Form and the QFT Circuit

5.1 The QFT as a product of single-qubit states

Write y in binary, $y = \Sigma_k\, y_k 2^k$, with $y_k \in \{0, 1\}$. Then

$$e^{2\pi i x y/2^n} = \Pi_{k=0}^{n-1} \left(e^{2\pi i x/2^{n-k}} \right)^{y_k}$$

Because the exponent splits into a product over bits of y, the sum over y factorises:

$$\mathrm{QFT}|x\rangle = \otimes_{k=0}^{n-1} \left(|0\rangle + e^{2\pi i x/2^{n-k}}|1\rangle\right)/\sqrt{2}$$

So the output is a **product state**: qubit k is in an equal superposition of $|0\rangle$ and $|1\rangle$ with a relative phase $2\pi x/2^{n-k}$ (qubit k = bit k of y; Qiskit's little-endian order). The QFT creates **no entanglement** when its input is a basis state. This factorisation is the key to an efficient circuit.

5.2 Building blocks

- **Hadamard (H):** turns a qubit with bit x_j into $(|0\rangle + (-1)^{x_j}|1\rangle)/\sqrt{2}$, which is the phase $e^{2\pi i \cdot x_j/2}$.
- **Controlled-phase gate R_k:** applies the phase $e^{2\pi i/2^k}$ to $|1\rangle$ only if the control is $|1\rangle$. It is diagonal: diag(1, 1, 1, $e^{2\pi i/2^k}$) on two qubits. Qiskit name: `cp(angle, control, target)`. $R_2 = \pi/2$, $R_3 = \pi/4$, and so on.
- **SWAP:** exchanges two qubits and is used at the end to put the qubits in the right order.

5.3 The algorithm for n qubits

```
for j from n-1 down to 0:
    apply H to qubit j
    for k from j-1 down to 0:
        apply controlled-phase with angle pi / 2^(j-k)
        (control = qubit k, target = qubit j)
finally: swap qubit i with qubit n-1-i for i = 0 .. n/2 - 1
```

Why it works. After H, qubit j carries the phase from its own bit x_j. Each controlled rotation adds the contribution of a lower bit x_k with the correct fraction $1/2^{j-k+1}$. So qubit j ends with phase $2\pi \cdot (0.x_j x_{j-1} \ldots x_0)$ in binary-fraction notation, which equals $2\pi x_{\le j}/2^{j+1}$. The product formula needs this phase on output qubit $n-1-j$, so a final reversal (swaps) places each qubit where it belongs.

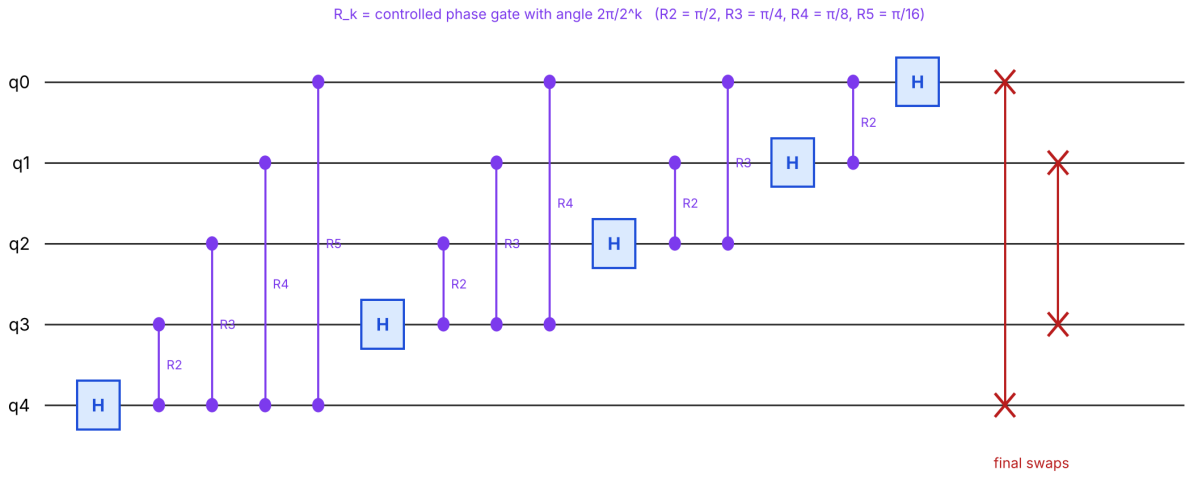


Figure 3: The 5-qubit QFT circuit built by the algorithm above (this is the structure that my_qft() in Section 11 produces).

5.4 Gate count

n qubits	H gates	Controlled-phase gates $n(n-1)/2$	Swaps $\lfloor n/2 \rfloor$	Classical FFT $\approx n \cdot 2^n$
5	5	10	2	160
10	10	45	5	10,240
20	20	190	10	$\approx 2.1 \times 10^7$

The QFT needs only $O(n^2)$ gates while the FFT needs $O(n \cdot 2^n)$ operations. I verified the counts for $n = 5$ by building the circuit (5 H, 10 cp, 2 swap).

6. Qiskit Bit Order (Little-Endian)

In Qiskit, qubit 0 is the **least significant** bit. A basis state with qubits (q_4 q_3 q_2 q_1 q_0) represents the integer $x = q_0 + 2q_1 + 4q_2 + 8q_3 + 16q_4$. Result strings are printed with the highest classical bit on the **left**, so '00110' means c4 c3 c2 c1 c0 = 0 0 1 1 0, which is the integer 6.

Preparation	Qubit values (q4 q3 q2 q1 q0)	Integer x
nothing	0 0 0 0 0	0
X on q0	0 0 0 0 1	1
X on q1	0 0 0 1 0	2
X on q1 and q0	0 0 0 1 1	3
X on q2	0 0 1 0 0	4

7. Analysis of Your Experiment: What Output Should We Expect?

7.1 The input state

```
qc.h(0)    # q0 : (|0> + |1>)/sqrt(2)
qc.x(1)    # q1 : |1>
```

Qubits 2, 3, 4 stay in $|0\rangle$. The 5-qubit state is $(|0\rangle_{q0} + |1\rangle_{q0})/\sqrt{2} \otimes |1\rangle_{q1} \otimes |000\rangle$. In integer form:

$$|\psi_{in}\rangle = (|2\rangle + |3\rangle)/\sqrt{2}$$

(q1 = 1 contributes 2; q0 = 0 or 1 contributes 0 or 1.)

7.2 Apply the QFT

$$\text{QFT}|\psi_{in}\rangle = (1/\sqrt{2})(1/\sqrt{32})\ \Sigma_y\ (\omega^{2y} + \omega^{3y})\ |y\rangle, \quad \omega = e^{2\pi i/32}$$

7.3 Probability of each outcome

The amplitude of $|y\rangle$ is $c_y = (\omega^{2y} + \omega^{3y})/\sqrt{64} = \omega^{2y}(1 + \omega^{y})/8$. Because $|\omega^{2y}| = 1$:

$$P(y) = |1 + \omega^{y}|^2/64 = (2 + 2\cos(2\pi y/32))/64 = \cos^2(\pi y/32)/16$$

$$P(y) = \cos^2(\pi y/32) \,/\, 16$$

(We used $2 + 2\cos\theta = 4\cos^2(\theta/2)$.) **Check:** $\Sigma_{y=0}^{31} \cos^2(\pi y/32) = 16$, so the probabilities add to 1. ✓

7.4 What the formula says

y	bits	P(y)	Comment
0	00000	6.25%	Largest ($\cos^2 = 1$)
1	00001	6.19%	
4	00100	5.33%	
8	01000	3.12%	$\cos^2(\pi/4) = 1/2$
12	01100	0.92%	
15	01111	0.06%	Almost never
16	10000	0%	Exactly zero: perfect destructive interference
17	10001	0.06%	Mirror of y = 15
24	11000	3.12%	Mirror of y = 8
31	11111	6.19%	Mirror of y = 1

- **Symmetry:** $P(y) = P(32 - y)$, because $\cos^2(\pi(32-y)/32) = \cos^2(\pi y/32)$. The distribution is symmetric around y = 16.
- **Hole at y = 16:** $\omega^{16} = -1$, so $1 + \omega^{16} = 0$. The two input terms cancel exactly.
- **Not uniform:** unlike a single basis state, the superposition of $|2\rangle$ and $|3\rangle$ interferes.

Physical meaning. The input is two neighbouring “positions” in superposition. Their Fourier contributions differ by a relative phase $\omega^{y} = e^{2\pi i y/32}$ that grows with y. Near y = 0 the two terms point in nearly the same direction and add (constructive interference); near y = 16 they point in

opposite directions and cancel (destructive interference).

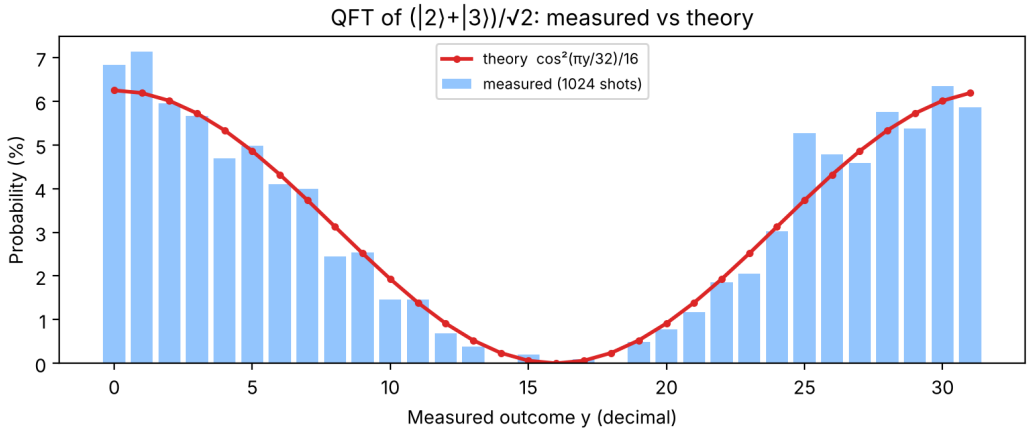


Figure 4: Measured counts (blue bars, 1024 shots) against the exact prediction (red curve).

Important
The measured percentages in Figure 4 come from one run and fluctuate. With 1024 shots each bar has statistical noise of a few tenths of a percentage point, so small deviations from the red curve are normal.

8. Your Code Explained Line by Line

8.1 Imports

```
from qiskit import QuantumCircuit, transpile
from qiskit.circuit.library import QFT
from qiskit_aer import AerSimulator
```

- `QuantumCircuit`: builds circuits.
- `transpile`: adapts the circuit to the simulator's gate set.
- `QFT`: circuit-library object for the QFT (deprecated in Qiskit 2.1+; see Section 10).
- `AerSimulator`: the simulator backend.

8.2 Registers

```
qc = QuantumCircuit(5, 5)
```

Five qubits (all |0⟩) and five classical bits to hold the measurement results.

8.3 Input state preparation

```
qc.h(0)
qc.x(1)
```

H puts q0 in (|0⟩+|1⟩)/√2; X flips q1 to |1⟩. Together they create (|2⟩ + |3⟩)/√2 as derived in Section 7.1.

8.4 Apply the QFT

```
qft = QFT(5)
qc.compose(qft, inplace=True)
```

`QFT(5)` creates a 5-qubit QFT circuit. `compose` appends it to the end of `qc` (acting on qubits 0 to 4), and `inplace=True` modifies `qc` directly instead of returning a new circuit. In the drawing, the QFT appears as one box labelled “QFT”.

8.5 Measure

```
qc.measure(range(5), range(5))
```

Qubit i is measured into classical bit i, for i = 0 to 4.

8.6 Simulate

```
sim = AerSimulator()
compiled_circuit = transpile(qc, sim)
result = sim.run(compiled_circuit, shots=1024).result()
```

Transpiling decomposes the QFT box into basic gates (H, controlled-phase, swap, ...). `shots=1024` repeats the whole circuit 1024 times, giving 1024 five-bit samples.

8.7 Output

```
print("QFT Results:", result.get_counts())
print(qc.draw())
```

`get_counts()` returns a dictionary {bitstring: count}. Each key is read c4 c3 c2 c1 c0, so '00110' is y = 6.

9. Complete Code and Real Output

Your program with line breaks restored, and with `sorted()` added so the dictionary prints in order. I ran it successfully:

```
# qft_basic.py : 5-qubit Quantum Fourier Transform demo
# Requires: pip install qiskit qiskit-aer
from qiskit import QuantumCircuit, transpile
from qiskit.circuit.library import QFT
from qiskit_aer import AerSimulator

# 1. Circuit with 5 qubits and 5 classical bits
qc = QuantumCircuit(5, 5)

# 2. Prepare the input state: q0 in (|0>+|1>)/sqrt(2), q1 in |1>, others |0>
qc.h(0)
qc.x(1)

# 3. Apply the 5-qubit QFT
qft = QFT(5)
qc.compose(qft, inplace=True)

# 4. Measure all qubits
qc.measure(range(5), range(5))

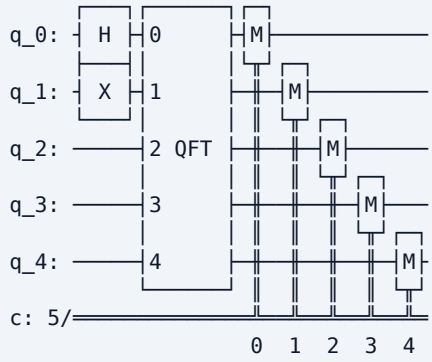
# 5. Simulate
sim = AerSimulator()
compiled_circuit = transpile(qc, sim)
result = sim.run(compiled_circuit, shots=1024).result()
counts = result.get_counts()

print("QFT Results:", dict(sorted(counts.items())))
print(qc.draw())
```

Output of one real run (1024 shots)

```
QFT Results: {'00000': 69, '00001': 64, '00010': 64, '00011': 61, '00100': 58,
 '00101': 42, '00110': 38, '00111': 32, '01000': 40, '01001': 22, '01010': 20,
 '01011': 14, '01100': 7, '01101': 2, '01110': 1, '10001': 1, '10010': 1,
 '10011': 3, '10100': 11, '10101': 11, '10110': 19, '10111': 28, '11000': 38,
 '11001': 36, '11010': 48, '11011': 63, '11100': 52, '11101': 65, '11110': 53,
 '11111': 61}
```

Circuit drawing



Reading the output. Counts are largest near 00000 and near 11111, fall towards the middle, and '10000' (y = 16) never appears. This matches Section 7: the shape is $\cos^2(\pi y/32)$, with an exact zero at 16. Also notice that the keys 01110, 10001 and 10010 each show a count of 1: those are the near-zero outcomes (0.24% theory = about 2.5 shots), seen only by chance.

Important
Running this on current Qiskit prints two `DeprecationWarning` messages about `QFT`. They are warnings, not errors.

10. Modern Code (QFTGate) Checked Against Theory

This version replaces the deprecated class with `QFTGate` and compares the measured frequencies with the exact formula $P(y) = \cos^2(\pi y/32)/16$. I ran it with qiskit 2.6.1 and qiskit-aer 0.18.2.

```
# qft_modern.py : same experiment with the current QFTGate, checked against theory
# Requires: pip install qiskit qiskit-aer numpy
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit.circuit.library import QFTGate
from qiskit_aer import AerSimulator

n = 5
N = 2 ** n

qc = QuantumCircuit(n, n)
qc.h(0)
qc.x(1)
qc.append(QFTGate(n), range(n))       # modern replacement for QFT(5)
qc.measure(range(n), range(n))

sim = AerSimulator()
shots = 1024
counts = sim.run(transpile(qc, sim), shots=shots).result().get_counts()

# ---- theory ------------------------------------------------------------
# Input: (|2> + |3>)/sqrt(2)   (q0 is the least significant bit, so x = q0 + 2*q1)
# P(y) = |(w^(2y) + w^(3y)) / sqrt(2N)|^2 = cos^2(pi*y/N) / (N/2)  with w = exp(2*pi*i/N)
w = np.exp(2j * np.pi / N)
amp = np.array([(w ** (2 * y) + w ** (3 * y)) / np.sqrt(2 * N) for y in range(N)])
p_theory = np.abs(amp) ** 2
print("sum of theoretical probabilities:", round(p_theory.sum(), 12))

print(" y  bits   theory%  measured%")
for y in range(N):
    key = format(y, f"0{n}b")
    meas = counts.get(key, 0) / shots
    print(f"{y:2d}  {key}  {100*p_theory[y]:7.2f}  {100*meas:8.2f}")

p_meas = np.array([counts.get(format(y, f"0{n}b"), 0) / shots for y in range(N)])
print("Total variation distance:", round(0.5 * np.abs(p_meas - p_theory).sum(), 4))
```

Output of one real run

```
sum of theoretical probabilities: 1.0
 y  bits   theory%  measured%
 0  00000     6.25      6.84
 1  00001     6.19      7.13
 2  00010     6.01      5.96
 3  00011     5.72      5.66
 4  00100     5.33      4.69
 5  00101     4.86      4.98
 6  00110     4.32      4.10
 7  00111     3.73      4.00
 8  01000     3.12      2.44
 9  01001     2.52      2.54
10  01010     1.93      1.46
11  01011     1.39      1.46
12  01100     0.92      0.68
13  01101     0.53      0.39
14  01110     0.24      0.00
15  01111     0.06      0.20
16  10000     0.00      0.00
17  10001     0.06      0.10
```

```
18  10010     0.24      0.00
19  10011     0.53      0.49
20  10100     0.92      0.78
21  10101     1.39      1.17
22  10110     1.93      1.86
23  10111     2.52      2.05
24  11000     3.12      3.03
25  11001     3.73      5.27
26  11010     4.32      4.79
27  11011     4.86      4.59
28  11100     5.33      5.76
29  11101     5.72      5.37
30  11110     6.01      6.35
31  11111     6.19      5.86
Total variation distance: 0.0495
```

Reading the result. The total variation distance (half the sum of absolute differences between measured and theoretical probabilities) is 0.05, which is what random sampling noise with 1024 shots over 32 outcomes typically produces. The measured values follow theory, including the exact zero at y = 16. This supports the claim that the circuit implements the QFT correctly.

> **Important**
> Your code is fine for a practical on a Qiskit version where `QFT` exists. If your lab computer has an older Qiskit that lacks `QFTGate`, use the original `QFT` version; if it has Qiskit 3.0 or later, use `QFTGate`. Since the exact version history is easy to get wrong, check your installed version with `pip show qiskit`.

11. Building the QFT Yourself and Verifying It

Writing your own QFT proves you understand it. The function below follows the algorithm in Section 5.3. It is then checked against Qiskit's gate and against the textbook DFT matrix.

```
# qft_manual.py : build the QFT yourself from H and controlled-phase gates
import numpy as np
from math import pi
from qiskit import QuantumCircuit
from qiskit.circuit.library import QFTGate
from qiskit.quantum_info import Operator

def my_qft(n, do_swaps=True):
    qc = QuantumCircuit(n, name=f"myQFT{n}")
    for j in reversed(range(n)):          # start from the most significant qubit
        qc.h(j)
        for k in range(j):                # controlled rotations from lower qubits
            qc.cp(pi / 2 ** (j - k), k, j)
    if do_swaps:                          # reverse qubit order
        for i in range(n // 2):
            qc.swap(i, n - 1 - i)
    return qc

n = 5
N = 2 ** n
mine = Operator(my_qft(n))

# 1) compare with Qiskit's own QFT
lib = QuantumCircuit(n); lib.append(QFTGate(n), range(n))
print("Equals Qiskit QFTGate:", mine.equiv(Operator(lib)))

# 2) compare with the textbook DFT matrix  F[y, x] = w^(x*y) / sqrt(N)
w = np.exp(2j * pi / N)
F = np.array([[w ** (x * y) for x in range(N)] for y in range(N)]) / np.sqrt(N)
print("Equals DFT matrix    :", np.allclose(mine.data, F))

# 3) unitary check and inverse check
U = mine.data
print("Unitary (U^dagger U = I):", np.allclose(U.conj().T @ U, np.eye(N)))
inv = Operator(my_qft(n).inverse())
print("QFT then inverse = identity:", np.allclose(inv.data @ U, np.eye(N)))

# 4) gate counts
c = my_qft(n)
print("Gate counts:", dict(c.count_ops()))
for m in (5, 10, 20):
    h = m; cp = m * (m - 1) // 2
    print(f"n={m:2d}: {h} H + {cp} controlled-phase gates (+ {m//2} swaps)")
```

Output of one real run

```
Equals Qiskit QFTGate: True
Equals DFT matrix    : True
Unitary (U^dagger U = I): True
QFT then inverse = identity: True
Gate counts: {'cp': 10, 'h': 5, 'swap': 2}
n= 5: 5 H + 10 controlled-phase gates (+ 2 swaps)
n=10: 10 H + 45 controlled-phase gates (+ 5 swaps)
n=20: 20 H + 190 controlled-phase gates (+ 10 swaps)
```

- **Equals Qiskit QFTGate:** the hand-built circuit has the same unitary matrix (`equiv` ignores a global phase).
- **Equals DFT matrix:** the 32×32 matrix equals $\omega^{xy}/\sqrt{32}$ entry by entry, confirming the definition in Section 4.

- **Unitary:** $U^\dagger U = I$, so it is a valid quantum operation.
- **Inverse:** `.inverse()` undoes the QFT. This is the form used in phase estimation and Shor's algorithm.

12. Period Finding: Where the QFT is Really Useful

The QFT detects **periodicity**. If the input state is a superposition of basis states spaced r apart, x = 0, r, 2r, ..., then after the QFT the probability is concentrated on multiples of N/r. This is the core of Shor's factoring algorithm.

Example. N = 32 and period r = 4: the input is (|0⟩ + |4⟩ + |8⟩ + ... + |28⟩)/√8. The multiples of 4 are the states with q0 = q1 = 0, so Hadamards on q2, q3, q4 create it. After the QFT, peaks are expected at y = 0, 8, 16, 24 (multiples of N/r = 8), each with probability 1/4.

Why: amplitude of |y⟩ is $(1/\sqrt{(8\cdot 32)})\ \Sigma_{m=0}^{7}\ \omega^{4my}$. This is a geometric sum of eighth roots of unity raised to y: it equals 8 when y is a multiple of 8 and 0 otherwise. So P(y) = 64/256 = 1/4 for y ∈ {0, 8, 16, 24}.

```
# qft_period.py : QFT reveals a hidden period (the idea behind Shor's algorithm)
from qiskit import QuantumCircuit, transpile
from qiskit.circuit.library import QFTGate
from qiskit_aer import AerSimulator

n = 5                                  # N = 32
qc = QuantumCircuit(n, n)

# Input state: equal superposition of x = 0, 4, 8, ..., 28   (period r = 4)
# x = 4m means q0 = q1 = 0, so put H on q2, q3, q4 only.
for q in (2, 3, 4):
    qc.h(q)

qc.append(QFTGate(n), range(n))
qc.measure(range(n), range(n))

sim = AerSimulator()
counts = sim.run(transpile(qc, sim), shots=2000).result().get_counts()
print("QFT of a period-4 state:")
for k, v in sorted(counts.items()):
    print(f"  y = {int(k, 2):2d}  ({k})  counts = {v}")
```

Output of one real run (2000 shots)

```
QFT of a period-4 state:
  y =  0  (00000)  counts = 508
  y =  8  (01000)  counts = 470
  y = 16  (10000)  counts = 523
  y = 24  (11000)  counts = 499
```

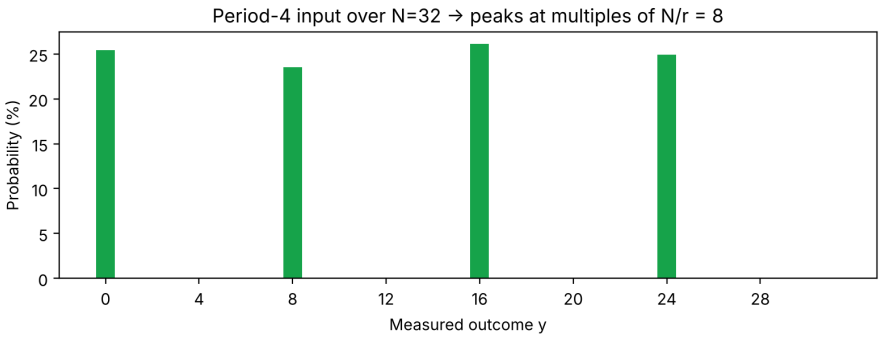


Figure 5: Only multiples of 8 appear, each about 25% of the time. The period 4 can be recovered as N/8 = 32/8.

Measuring any one peak gives a multiple of N/r. A few repetitions, plus a bit of classical number theory (greatest common divisors), reveal r. In real Shor, the period belongs to a function such as a^x mod M, which needs additional circuitry to prepare; this demo shows only the QFT stage.

13. Applications and Honest Limitations

Algorithm / task	Role of the QFT
Shor's factoring algorithm	Finds the period of a^x mod M, from which factors are obtained
Quantum phase estimation (QPE)	The inverse QFT converts phase information into a measurable bitstring
Hidden subgroup problems	Extracts the hidden structure of a function
Quantum chemistry / simulation	QPE reads out energy eigenvalues
Linear-systems algorithm (HHL)	Uses QPE internally

13.1 Limitations (important to state in a viva)

- **The QFT does not give you the Fourier coefficients.** They are the 2^n amplitudes of the output state, and measuring returns only one sample y with probability $|\text{amplitude}|^2$. You cannot read all N values, which is why the QFT cannot simply replace the FFT for signal processing.
- **Gate-count comparison is not a like-for-like speedup.** The QFT circuit is small ($O(n^2)$ gates) but gives access only to measurement statistics, while the FFT outputs all N numbers.
- **The speedup appears inside larger algorithms** (such as Shor's), where the surrounding steps arrange for the final measurement to be informative.
- **Your demo is educational:** a 5-qubit QFT is easily simulated classically.

14. Troubleshooting

Problem	Cause	Fix
DeprecationWarning about QFT / BlueprintCircuit	QFT class deprecated since Qiskit 2.1	Use QFTGate: qc.append(QFTGate(5), range(5))
ImportError for QFT after upgrading to Qiskit 3	Class removed	Switch to QFTGate
ImportError for QFTGate	Older Qiskit without it	Upgrade (pip install -U qiskit) or use the QFT class
Counts look “symmetric” and reversed	Little-endian bit order	Remember q0 is the right-most character
Uniform output over all 32 values	Input was a single basis state	Use a superposition (Section 4.2)
Bitstring I expect never appears	Probability is exactly or nearly zero (e.g. y = 16)	Check against P(y) = $\cos^2(\pi y/32)/16$
Output differs from a textbook DFT sign	Sign convention of exponent	Qiskit's QFT uses e^{+2πixy/N}
Keys missing from counts dict	Outcomes with zero counts are omitted	Use counts.get(key, 0)

15. Viva Questions and Answers

Q1. What is the Quantum Fourier Transform?

A unitary operation that maps $|x\rangle$ to $(1/\sqrt{N})\ \Sigma_y\ e^{2\pi i xy/N}|y\rangle$. It applies the discrete Fourier transform to a state's amplitudes.

Q2. How is it related to the classical DFT/FFT?

It performs the same linear map (with $1/\sqrt{N}$ normalisation) on the amplitude vector, but it is implemented as a circuit on n qubits for $N = 2^n$.

Q3. What is a root of unity?

A complex number z with $z^N = 1$; here $\omega = e^{2\pi i/N}$.

Q4. What is the 1-qubit QFT?

The Hadamard gate.

Q5. What gates make up the QFT circuit?

Hadamard gates, controlled-phase rotations R_k and final swaps.

Q6. How many gates does an n-qubit QFT use?

n Hadamards, $n(n-1)/2$ controlled-phase gates and $\lfloor n/2 \rfloor$ swaps, so $O(n^2)$.

Q7. Why are there swaps at the end?

The circuit produces the output qubits in reversed order; swaps restore the standard ordering.

Q8. What does a controlled-phase gate do?

Multiplies the $|11\rangle$ component by $e^{i\theta}$; with $\theta = 2\pi/2^k$ it is R_k.

Q9. Is the QFT output entangled?

Not for a basis-state input: it is a product of single-qubit states (Section 5.1).

Q10. What is the inverse QFT?

The adjoint circuit, using ω^{-xy}; in Qiskit, .inverse().

Q11. What state does your code prepare before the QFT?

$(|2\rangle + |3\rangle)/\sqrt{2}$, since q0 is in superposition and q1 is $|1\rangle$ (little-endian).

Q12. What is the theoretical output distribution for your code?

$P(y) = \cos^2(\pi y/32)/16$.

Q13. Why is y = 16 never observed?

$\omega^{16} = -1$, so the contributions of $|2\rangle$ and $|3\rangle$ cancel exactly (destructive interference).

Q14. Why would a single basis state give a uniform distribution?

Every amplitude has magnitude $1/\sqrt{N}$; the information lies in the phases, which a computational-basis measurement cannot see.

Q15. What is the Fourier basis?

The set of states $QFT|x\rangle$, which differ in phase only.

Q16. Which Qiskit function adds the QFT, and what changed recently?

qiskit.circuit.library.QFT, which is deprecated since Qiskit 2.1 in favour of QFTGate.

Q17. What does qc.compose(qft, inplace=True) do?

Appends the QFT circuit to qc and modifies qc directly.

Q18. Why do we transpile?

To rewrite the circuit using gates the simulator or device supports.

Q19. What does shots=1024 mean?

The circuit is run 1024 times and each run gives one measured bitstring.

Q20. What is the bit order in Qiskit results?

Little-endian: the right-most character is classical bit 0.

Q21. Can the QFT replace the FFT?

No for general data: the transformed values are amplitudes that cannot all be read out. Its power lies in algorithms such as Shor's and phase estimation.

Q22. What does the QFT do to a periodic input?

Concentrates probability on multiples of N/r, where r is the period.

Q23. Where is the QFT used in Shor's algorithm?

To extract the period of the modular exponentiation function from a superposition.

Q24. What is the QFT matrix for N = 4?

½ times the matrix with rows (1,1,1,1), (1,i,−1,−i), (1,−1,1,−1), (1,−i,−1,i).

16. Glossary

Term	Definition
Amplitude	Complex number whose squared magnitude is a measurement probability
Controlled-phase (CP) gate	Applies a phase to $\|1\rangle$ of the target only if the control is $\|1\rangle$
Destructive / constructive interference	Cancellation / reinforcement of amplitudes when they are added
DFT / FFT	Discrete Fourier transform / its fast algorithm
Euler's formula	$e^{i\theta} = \cos\theta + i\sin\theta$
Fourier basis	Basis consisting of the states $QFT\|x\rangle$
Frequency domain	Description of a signal by its frequency content
Gate (Qiskit) vs circuit	A single operation vs a sequence of operations; QFTGate is a single gate
Inverse QFT	$QFT^\dagger$; maps Fourier-basis states back to computational states
Little-endian	Least significant bit stored first (qubit 0)
Period	Spacing r at which a pattern repeats
Phase	Angle of a complex amplitude
Phase estimation	Algorithm that determines an eigenvalue phase using the inverse QFT
Product state	State that factorises into single-qubit states
Root of unity	Complex number z with $z^N = 1$
Shots	Number of repeated circuit executions
Total variation distance	Half the sum of absolute differences between two probability distributions
Unitary	Reversible operation: $U^\dagger U = I$

17. Result and Conclusion

A 5-qubit QFT was implemented in Qiskit and applied to the input state $(|2\rangle + |3\rangle)/\sqrt{2}$. The measured distribution follows the exact prediction $P(y) = \cos^2(\pi y/32)/16$, including the exact zero at $y = 16$, with a total variation distance of about 0.05 from theory at 1024 shots. A hand-built QFT from Hadamard and controlled-phase gates matched both Qiskit's gate and the DFT matrix, and a period-4 input produced peaks at multiples of 8, as predicted.

Key concept to remember: the QFT is the discrete Fourier transform acting on amplitudes. It stores information in phases, which measurement alone cannot see; its value comes from interference inside larger algorithms.

Five topics to focus on for your exam

1. QFT definition and its 4×4 matrix example
2. Roots of unity and phase ($e^{i\theta}$)
3. Circuit structure: H, R_k controlled-phase gates, swaps, and the $O(n^2)$ gate count
4. Little-endian bit order and the derivation of the input state $(|2\rangle+|3\rangle)/\sqrt{2}$ with $P(y) = \cos^2(\pi y/32)/16$
5. Why a basis state gives a flat distribution, and how periodicity produces peaks